Image Similarity Search Indexing System

Name: Yerraiahgari Ram Charan Goud
Roll No: 241212
Project: IITK ACA Summer Project - GanForge

This document details the workflow for building an Approximate Nearest Neighbors (Annoy) index using ResNet18 feature embeddings to retrieve visually similar images.

1. Model Initialization and Feature Extraction Setup

```
import torch
import torch.nn as nn
from torchvision import models, transforms
from annoy import AnnoyIndex
from PIL import Image

weights = models.ResNet18_Weights.IMAGENET1K_V1
model = models.resnet18(weights=weights)
model.fc = nn.Identity()
model.eval()
```

- **ResNet18 Architecture**: Loads a pre-trained ResNet18 model using ImageNet weights.
- **Feature Embedding**: By overriding the final fully connected layer (`model.fc = nn.Identity()`), the network is repurposed from a classifier into a feature extractor, outputting a 512-dimensional vector representation for any given image.
- **Evaluation Mode**: `model.eval()` ensures layers like batch normalization and dropout behave deterministically for inference.

2. Image Preprocessing

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor()
])
```

Incoming images are standardized to a 224x224 spatial resolution and converted to PyTorch tensors, matching the expected input format of the ResNet18 backbone.

3. Building the Annoy Index (SASIS Phase)

```
annoy_index = AnnoyIndex(512, 'angular')

for i in range(len(images)):
    image = Image.open(os.path.join(images_folder, images[i]))
    input_tensor = transform(image).unsqueeze(0)

    if input_tensor.size()[1] == 3:  # Ensure RGB format
        output_tensor = model(input_tensor)
        annoy_index.add_item(i, output_tensor[0])

annoy_index.build(10)
annoy_index.save('dog_index.ann')
```

- **Annoy Initialization**: A search index is instantiated for 512-dimensional vectors using the 'angular' distance metric (cosine similarity).
- **Batching**: `unsqueeze(0)` adds a necessary batch dimension to the single image tensor.
- **Indexing**: Feature vectors are extracted and iteratively added to the Annoy structure. The index is then built using 10 trees (balancing precision and memory) and serialized to disk as `dog_index.ann`.

4. Querying and Visualizing Results (FSI Phase)

```
annoy_index.load('dog_index.ann')
image_grid = Image.new('RGB', (1000, 1000))

# For a given query image tensor:
output_tensor = model(input_tensor)
nns = annoy_index.get_nns_by_vector(output_tensor[0], 24)

# Bounding box for the query image
image = image.resize((200, 200))
image_draw = ImageDraw.Draw(image)
image_draw.rectangle([(0, 0), (199, 199)], outline='red', width=8)
image_grid.paste(image, ((0, 0)))

# Appending the 24 nearest neighbors
for j in range(24):
```

```
    search_image = Image.open(os.path.join(images_folder, images[nns[j]]))
    search_image = search_image.resize((200, 200))
    image_grid.paste(search_image, (200 * ((j + 1) % 5), 200 * ((j + 1) //
5)))

image_grid.save(f'ImageDump/image_{i}.png')
```

- **Retrieval**: The saved index is loaded. The `get_nns_by_vector` method rapidly retrieves the indices of the 24 closest matching feature vectors.
- **Grid Construction**: A blank 1000x1000 canvas is generated. The original query image is highlighted with a red bounding box and placed in the top-left corner.
- **Population**: The 24 visually similar neighbor images are dynamically resized to 200x200 and mathematically tiled into the remaining slots of the 5x5 grid. The final collage is exported to the local file system.

Note on Outputs: The final executed script will output 5x5 image grids into the `ImageDump/` directory, visualizing the query dog alongside its closest algorithmic matches.